

TÓPICO 00 — PRIMEIRA ETAPA

Introdução à Disciplina

O que é Backend, como a disciplina funciona e o que vamos construir juntos

Bem-vindo(a) ao mundo do Backend

Vamos explicar cada termo novo na primeira vez em que ele aparecer, sem pressupor que você já conhece nada de programação.

O que você vai conseguir fazer ao final

Construir sozinho(a) um sistema completo: uma API que recebe pedidos de um app ou site, processa regras de negócio e guarda dados em um banco de dados.

Como vamos chegar lá

Passo a passo, uma camada por vez — primeiro o ambiente de trabalho, depois como a Web funciona, depois a linguagem Java, e só então APIs de verdade.

O que fazer se algo não fizer sentido

Se um termo técnico não foi explicado, é sinal de que eu preciso parar e explicar — não de que você está atrasado(a).

Quem vai te acompanhar nessa jornada

Prof. Daniel Plácido

Trabalho com backend desde 2002. Trabalhei com Java, NodeJS, Flutter, ReactJS, ReactNative, Caché Intersystems, Cobol, JavaScript, ETC.... Por encrível que pareça, um pouco de Python...

Bancos de dados: Oracle, Postgres, MySQL, MariaDB, Redis, Caché Intersystems, SQL Server.

Trabalho Diretamente com Docker, Git, AWS, Google GPC, Azure, etc.

Serviços de Message Queue: Redis, BullMQ, RabbitMQ, ActiveMQ Apache, Amazon SQS, etc.

Ja trabalhei coordenando equipes com mais de 50 pessoas entre DEVs, Testers, Analistas de negócio, Engenheiros de Software, Designers, etc.

Trabalhei home-office para uma empresa dos EUA, recentemente.

Formado em Ciências da Computação e Pos Graduada em Gerenciamento de Projetos.

Atualmente cursando uma Pos-graduação em Engenharia de IA aplicada com foco em backend.

Meu objetivo aqui não é só ensinar sintaxe de código, é ensinar a pensar como um desenvolvedor backend resolve problemas e preparar você para o mercado.

O que é, afinal, "programar"?

A analogia da receita de bolo

Uma receita é uma sequência de passos que, seguida na ordem certa, sempre produz o mesmo resultado: "quebre 2 ovos", "misture com 1 xícara de farinha", "leve ao forno por 40 minutos".

Programar é escrever uma receita — chamada de código — que o computador executa passo a passo, à risca, sem interpretar ou pular etapas.

A diferença: o computador é extremamente literal. Se a receita disser "leve ao forno" mas você não disse antes "ligue o forno", ele não vai adivinhar.

Traduzindo os termos

Código: o texto com as instruções, escrito em uma linguagem que o computador entende (ex: Java).

Programa / sistema: o conjunto de instruções já pronto, rodando.

Executar / rodar: o computador está, agora, seguindo as instruções.

Bug: um passo da receita que estava errado e deu um resultado inesperado.

Frontend e Backend: a analogia do restaurante

Todo sistema com o qual você interage — um app, um site — é dividido em duas grandes partes que trabalham juntas, como um restaurante:

O SALÃO — Frontend

É a parte que o cliente vê e toca: a mesa, o cardápio, a decoração, o garçom.

No mundo digital: as telas, os botões, as cores, os formulários — tudo que você vê e clica em um app ou site.

Exemplos: a tela de login do Instagram, o carrinho de compras de um e-commerce.

A COZINHA — Backend

É a parte que o cliente NÃO vê: o cozinheiro, o estoque de ingredientes, as receitas, as regras da casa ("esse prato só é servido depois das 18h").

No mundo digital: o servidor que recebe o pedido, aplica as regras de negócio e guarda/busca informações no banco de dados.

É essa parte que vamos estudar durante todo este semestre.

O que exatamente o Backend faz?

1

Recebe pedidos

Escuta as solicitações que chegam do frontend (do app/site) — ex.: "quero fazer login", "quero ver meus pedidos".

2

Aplica regras de negócio

Decide o que é permitido: "esse cupom só vale para compras acima de R\$50", "esse usuário não pode ver dados de outro usuário".

3

Conversa com o banco de dados

Guarda informação de forma permanente (cadastro de um cliente) e busca informação quando necessário (histórico de pedidos).

4

Devolve uma resposta

Envia de volta ao frontend o resultado: os dados pedidos, uma confirmação, ou uma mensagem de erro explicando o que deu errado.

ONDE O BACKEND JÁ ESTÁ, SEM VOCÊ PERCEBER

Backend no seu dia a dia

iFood

Quando você faz um pedido, o backend verifica se o restaurante está aberto, calcula o valor da entrega, reserva o pedido na fila da cozinha e avisa o entregador.

Instagram

Quando você curte uma foto, o backend registra a curtida, atualiza o contador e decide quem vai ver essa curtida no feed.

Netflix

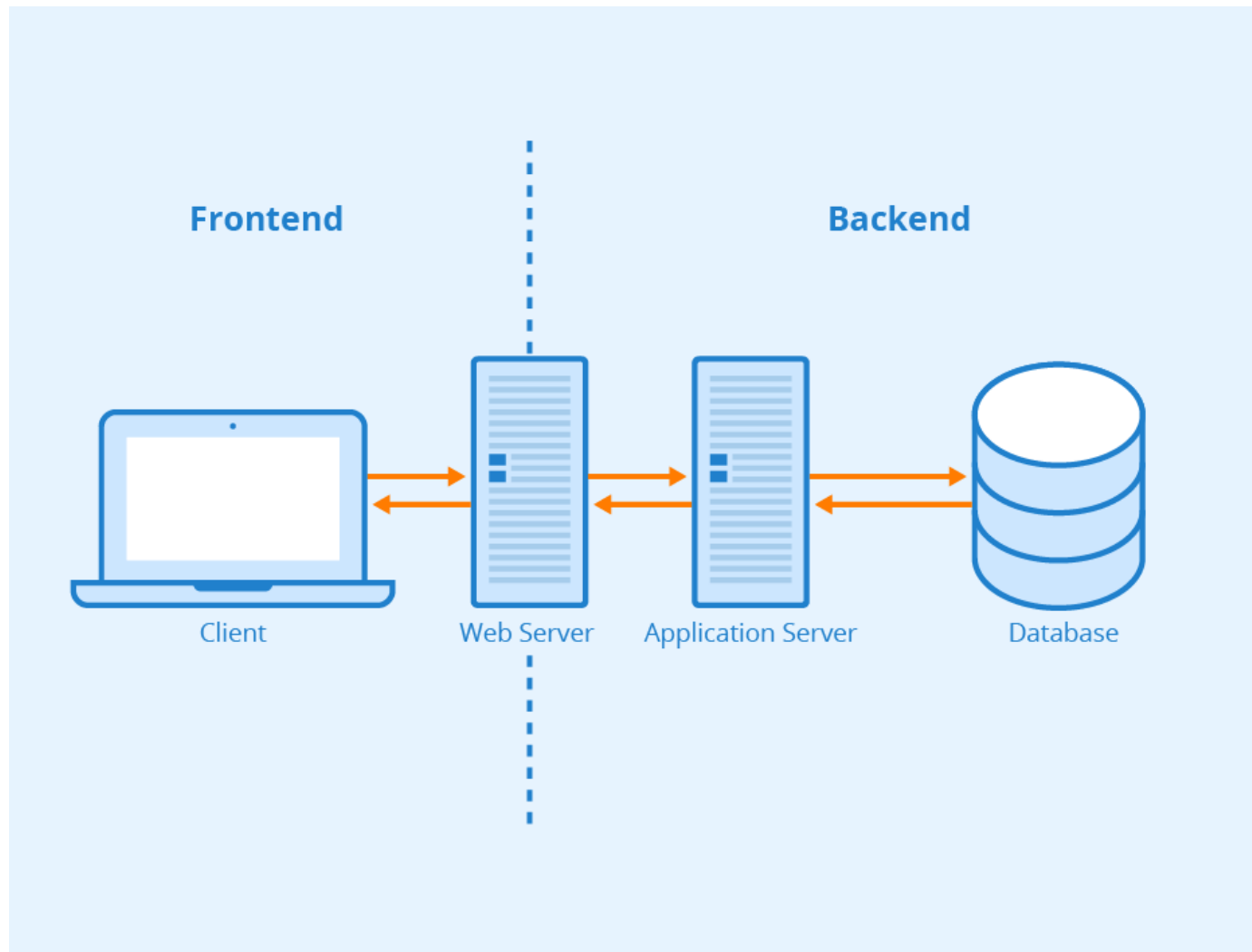
O backend sabe em que minuto você parou de assistir, sugere séries parecidas e libera o vídeo só para quem tem assinatura ativa.

Home Banking

O backend confere seu saldo, valida a senha, aplica as regras de segurança contra fraude e só então autoriza a transferência.

ONDE O BACKEND JÁ ESTÁ, SEM VOCÊ PERCEBER

Backend no seu dia a dia



Por que investir tempo em Backend?

Alta demanda

Toda empresa que tem um app, site ou sistema interno precisa de alguém cuidando do que roda "por trás".

Trabalho remoto

É uma das áreas de TI com mais oportunidades de trabalho remoto e para empresas fora da sua cidade.

Base sólida

As mesmas lógicas (API, banco de dados, regras de negócio) se repetem em quase todo sistema — aprender uma vez, aplicar em muitos contextos.

Evolução constante

Dev backend raramente para de aprender — e isso é bom: o mercado valoriza quem se atualiza continuamente.

Ementa: as 3 grandes etapas

1ª Etapa

Ambiente de Desenvolvimento

Fundamentos da Web (como a internet funciona), protocolo HTTP, e a linguagem Java: variáveis, orientação a objetos, lógica de programação.

2ª Etapa

API REST

Como sistemas conversam entre si: arquitetura REST, persistência de dados em banco de dados, DTOs, e a construção de uma API completa em Java com Spring.

3ª Etapa

Tópicos Avançados

Integração de serviços, observabilidade e testes, Model Context Protocol (MCP), autenticação e segurança, infraestrutura — e o projeto final.

Ambiente de Desenvolvimento

Arquitetura da Web e Protocolo HTTP

Como navegador e servidor conversam: o que é uma requisição, o que é uma resposta, o que são status codes (200, 404, 500...).

Orientação a Objetos em Java

A forma de organizar código que vamos usar durante todo o curso: classes, objetos, atributos e métodos, explicados com analogias do mundo real.

Tópicos especiais e ambiente de desenvolvimento

Instalação e configuração das ferramentas (JDK, IDE, Git, Kilocode AI) que você vai usar em toda a disciplina.

Avaliação N1

Primeira prova, cobrindo os fundamentos vistos até aqui.

API REST

Web Services e padrões de arquitetura

O que é uma API, por que ela existe, e os principais estilos de arquitetura usados no mercado.

Arquitetura REST e persistência de dados

As regras que tornam uma API "REST", e como salvar/consultar informações em um banco de dados de verdade.

DTO e mapeamento

Como organizar os dados que entram e saem da sua API de forma segura e organizada.

Avaliação N2a e Projeto Intermediário N2b

Uma prova teórica (N2a) e um projeto prático (N2b) em que você constrói uma API própria, do zero ao funcionamento completo.

Tópicos Avançados

Integração de serviços

Como sua API conversa com outros sistemas e serviços externos (ex.: envio de e-mail, gateway de pagamento).

Observabilidade e testes

Como saber se o seu sistema está funcionando bem em produção, e como escrever testes automáticos que garantem isso.

Model Context Protocol (MCP)

Um protocolo novo que permite que sistemas de Inteligência Artificial, como o Claude, se conectem e usem ferramentas de forma padronizada.

Autenticação, segurança e infraestrutura

Como proteger sua API contra acessos indevidos, e onde/como ela roda "no mundo real" (servidores, nuvem).

Projeto Final e Avaliação N3

Você constrói, sozinho(a) ou em grupo, um sistema completo aplicando tudo que foi aprendido no semestre.

Cronograma completo — 19 aulas

1	Introdução à Disciplina + Arquitetura da Web
2	Ambiente de Desenvolvimento + Kilocode AI
3-4	Orientação a Objetos em Java (parte I e II)
5	Avaliação N1
6	Web Services + Protocolo HTTP/JSON
7	Padrões de Arquitetura + Arquitetura REST
8	Persistência de Dados
9	DTO e Mapeamento
10	Avaliação N2a

11	Projeto Intermediário N2b
12	Integração de Serviços
13	Observabilidade e Testes
14	Model Context Protocol (MCP) + IA no backend
15	Autenticação e Segurança
16	Infraestrutura
17-18	Desenvolvimento do Projeto Final
19	Projeto Final — Avaliação N3

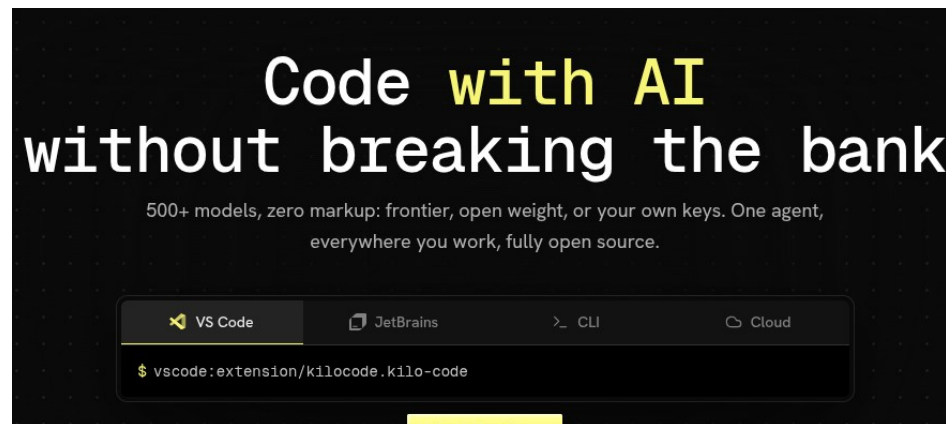
Datas específicas a definir — cronograma sujeito a ajustes conforme o andamento da turma.

Uso de IA

Você pode utilizar Inteligência Artificial no desenvolvimento das atividades. Porém, é indispensável compreender a teoria, pois ela é a base para aplicar corretamente os conceitos e revisar criticamente o código gerado pela IA. Saber utilizar IA com conhecimento técnico é uma competência cada vez mais exigida pelo mercado de trabalho. Provas e avaliações NÃO podem utilizar IA!!!!!!

Vamos utilizar o kilocode.ai

Durante o curso, utilizaremos o KiloCode, uma ferramenta gratuita para desenvolvimento de software que permite utilizar diversos modelos de Inteligência Artificial por meio do OpenRouter.ai. Ela auxiliará na geração, explicação e revisão de código, proporcionando uma experiência prática semelhante à encontrada no mercado de trabalho.



Como você vai ser avaliado(a)

N1

Prova teórica

Cobre Arquitetura da Web, HTTP e Orientação a Objetos em Java. Formato prova escrita/individual.

N2a

Prova teórica

Cobre Web Services, REST e Persistência de Dados.

N2b

Projeto prático

Você constrói uma API REST funcional, aplicando o que foi visto até aqui — é aqui que a teoria vira prática de verdade.

N3

Projeto Final

Sistema completo, integrando todos os tópicos avançados vistos na 3ª etapa.

O LabTec

O LabTec é o laboratório de tecnologia da SATC — o espaço físico onde as aulas práticas de programação acontecem.

Computadores prontos para programar

Já vêm com as ferramentas básicas configuradas, então você não perde tempo de aula instalando programas.

Aprendizado dinâmico

Aprende praticando

Regras de uso

Chegue no horário, salve seu trabalho antes de sair (usaremos o Git/GitHub para isso — explicado no próximo slide) e mantenha o espaço organizado para a próxima turma.

Onde encontrar todo o material: o repositório no GitHub

O que é o GitHub?

Pense no GitHub como um Google Drive feito especificamente para guardar código e arquivos de projetos de programação, com histórico de todas as versões.

Um "repositório" (ou "repo") é, nesse contexto, equivalente a uma "pasta" — um espaço onde ficam organizados todos os arquivos de um mesmo projeto ou disciplina.

Tudo que usarmos em aula — slides, materiais de apoio, exemplos de código — vai ficar disponível no repositório da disciplina, sempre atualizado.

Como acessar

1. Acesse github.com/placidodaniel no navegador (não precisa de conta para apenas visualizar).
2. Localize a pasta da disciplina de Backend e, dentro dela, a pasta do tópico da aula.
3. Baixe os arquivos (botão "Download" ou "Raw") ou apenas leia diretamente pelo navegador.
4. Sempre que uma pasta for atualizada, o material mais novo estará ali — não precisa pedir por e-mail.

ANTES DE COMEÇAR

Ferramentas que você vai ouvir bastante — em 1 frase cada

JDK

O "motor" que traduz e executa código Java no seu computador.

IDE (ex.: IntelliJ)

Um editor de texto avançado, feito especialmente para escrever código com ajuda automática.

Git

Uma ferramenta que guarda o histórico de todas as versões do seu código, permitindo voltar atrás se algo quebrar.

GitHub

Um site que hospeda repositórios Git na nuvem, permitindo compartilhar e colaborar em projetos.

Terminal

Uma tela de texto onde você digita comandos diretamente para o computador, em vez de clicar em ícones.

API

Um "cardápio" de operações que um sistema oferece para outros sistemas conversarem com ele.

Banco de dados

Um sistema especializado em guardar informação de forma organizada e permanente.

Postman

Um programa usado para testar APIs manualmente, simulando pedidos antes de ter um frontend pronto.

Combinados de sala e comunicação

Chegue na hora (início as 18h50 às 22h00, com intervalo das 20h10 às 20h20)

Você reprova por faltas, 5 faltas é o suficiente para reprovar. Atestados médicos: Alunos devem enviar para eng.software@satc.edu.br. Entretanto: O atestado não abona falta, mas permite agendamento de avaliação de 2ª chamada, se entregue via e-mail em até 48h após a avaliação perdida... 1ª chamada: Entre 19h10 e 19h20; 2ª chamada: Após as 21h30.

Sou legal, mais sou justo!

Eu ajudo a todos! Mais cobro também! Dois projetos iguais ou copiado de alguma fonte, reprovado!

Uso de IA – Pode usar!

IA precisa ser usado para entendermos como funciona a dinâmica de desenvolvimento utilizando IA. O que será avaliado nos projetos e provas será o conhecimento e aplicação das técnicas apresentadas.

Canal oficial de comunicação

Toda comunicação ocorrerá pelo portal da SATC, parte de mensagens, e-mail: daniel.placido@satc.edu.br. Todo material da aula esta no GitHub: <https://github.com/placidodaniel/backend-aulas-satc>. Projetos devem subir para o GitHub de maneira publica e enviar o link para o e-mail daniel.placido@satc.edu.br.

RESUMO DA AULA

O que levamos de hoje

- Backend é a "cozinha" do sistema: regras de negócio, processamento e banco de dados.
- A disciplina tem 3 etapas: Ambiente de Desenvolvimento → API REST → Tópicos Avançados.
- Avaliação combina provas teóricas (N1, N2a) com projetos práticos (N2b, N3).
- Todo material fica disponível e sempre atualizado no repositório do GitHub.

PRÓXIMA AULA — TÓPICO 01

Introdução à Web e Arquitetura da Web — de onde vem a internet que usamos todos os dias.